

Mbeya University of Science and Technology

CS 8119/ET 8108: Introduction to High Level Programming

Facilitator: Mr. Magemo A

Email: magemohoward30@gmail.com



shutterstock.com • 1680857539



Grading System

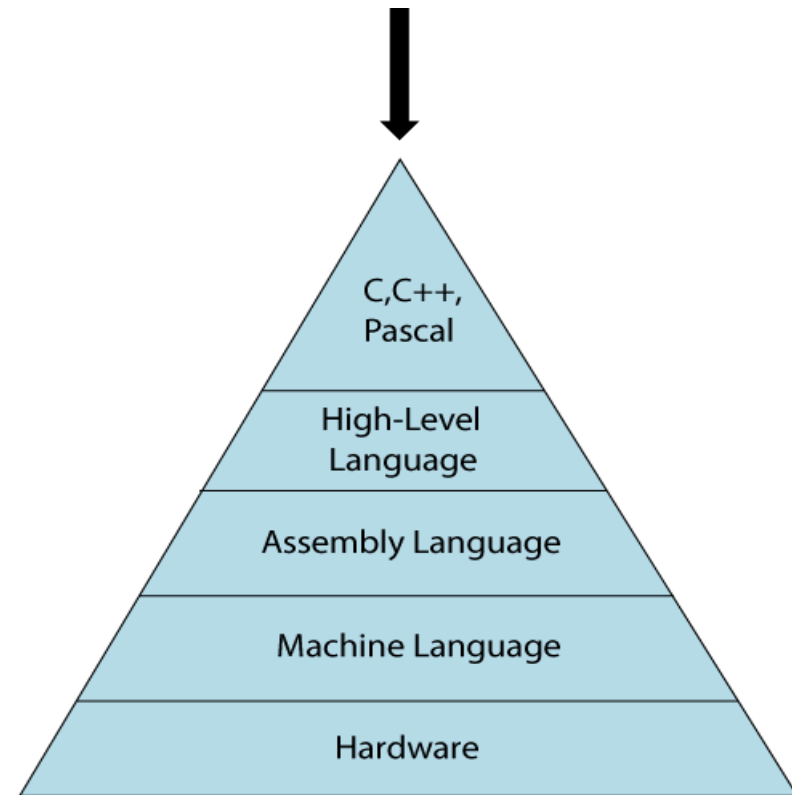
Test#1 (Week 6/7) -----	15%
Test#2 (Week 11/12)-----	15%
Quiz-----	5%
Assignments -----	5%
University Examination -----	60%

◆ Introduction

- The high-level language is a programming language that allows a programmer to write the programs which are independent of a particular type of computer.



The high-level languages are considered as high-level because they are closer to human languages than machine-level languages.





Introduction

A **compiler** is required to translate a high-level language into a low-level language.

Advantages of a high-level language

1. The high-level language is easy to **read, write, and maintain** as it is written in English like words.
2. The high-level languages are designed to overcome the limitation of low-level language, i.e., portability. **The high-level language is portable**; i.e., these languages are machine-independent.



Introduction

Types of high-level language that are commonly used today

1. Python.
2. Java.
3. C++
4. C#
5. Visual Basic.
6. JavaScript.



Introduction

In this course C++ Will be discussed



Introduction

What is C++?

- C++ is an object-oriented programming language. It is an extension to C programming.
- C++ supports the object-oriented programming, the four major pillar of object-oriented programming (OOPs) used in C++ are:
 1. Inheritance
 2. Polymorphism
 3. Encapsulation
 4. Abstraction



Introduction

C++ Standard Libraries

Standard C++ programming is divided into three important parts:

1. The core library includes the data types, variables and literals, etc.
2. The standard library includes the set of functions manipulating strings, files, etc.
3. The Standard Template Library (STL) includes the set of methods manipulating a data structure.



Introduction

Application of C++

By the help of C++ programming language, we can develop different types of secured and robust applications:

1. Window application
2. Client-Server application
3. Device drivers
4. Embedded firmware etc



Introduction

C++ Features

1. Simple

- C++ is a simple language because it provides a structured approach (to break the problem into parts)

2. Abstract Data types

- In C++, complex data types called Abstract Data Types (ADT) can be created using classes.

3. Portable

- C++ is a portable language and programs made in it can be run on different machines.



Introduction

1. Structured programming language

- C++ is a structured programming language. In this we can divide the program into several parts using functions.

2. Rich Library

- C++ provides a lot of inbuilt functions that make the development fast.

Following are the libraries used in C++ programming are:

<iostream>

<cmath>

<cstdlib>

<fstream>



Introduction

1. Pointer

- C++ provides the feature of pointers. We can use pointers for memory, structures, functions, array, etc.

2. Recursion

- In C++, we can call the function within the function. It provides code reusability for every function.

3. Reusability

- With the use of inheritance of functions programs written in C++ can be reused in any other program of C++

- 4. **Errors are easily detected:** It is easier to maintain a C++ programs as errors can be easily located and rectified



Introduction

C++ Program

- Open the C++ console and write the following code:

```
#include <iostream>
using namespace std;
int main()
{
    cout << "Welcome to C++ Programming.";
    return 0;
}
```



Introduction

#include<iostream> includes the standard input output library functions. It provides cin and cout methods for reading from input and writing to output respectively.

Using namespace std; it is a collection of identifiers that belong to a group or family. Eg cout belong to std library

Int main() The main() function is the entry point of every program in C++ language. The void keyword specifies that it returns no value.



Introduction

`cout << "Welcome to C++ Programming."` is used to print the data "Welcome to C++ Programming." on the console.



Introduction

Basic Input/Output

I/O operation is using the stream concept. **Stream is the sequence of bytes or flow of data.** It makes the performance fast.

- If bytes flow from main memory to device like printer, display screen, or a network connection, etc, this is called as **output operation.**
- If bytes flow from device like printer, display screen, or a network connection, etc to main memory, this is called as **input operation.**



Introduction

Basic Input/Output

Header File	Function and Description
<code><iostream></code>	It is used to define the <code>cout</code> , <code>cin</code> and <code>cerr</code> objects, which correspond to standard output stream, standard input stream and standard error stream, respectively.
<code><iomanip></code>	It is used to declare services useful for performing formatted I/O, such as <code>setprecision</code> and <code>setw</code> .
<code><fstream></code>	It is used to declare services for user-controlled file processing.



Introduction

Standard output stream (cout)

- The cout is a predefined object of ostream class.
- It is connected with the standard output device, which is usually a display screen.
- The cout is used in conjunction with stream insertion operator (<<) to display the output on a console



Introduction

Example of standard output stream (cout):

```
#include <iostream>

using namespace std;

int main() {
    char ary[] = "Welcome to C++ tutorial";
    cout << "Value of ary is: " << ary << endl;
}
```

Output:

Value of ary is: [Welcome to C++ tutorial](#)



Introduction

Standard input stream (cin)

- The cin is a predefined object of istream class.
- It is connected with the standard input device, which is usually a keyboard.
- The cin is used in conjunction with stream extraction operator (>>) to read the input from a console.



Introduction

Example of standard input stream (cin):

```
#include <iostream>

using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;
    cout << "Your age is: " << age << endl;
}
```

Output:

Enter your age: 22

Your age is: 22



Introduction

Standard end line (endl)

- The endl is a predefined object of ostream class.
- It is used to insert a new line characters and flushes the stream.

```
#include <iostream>

using namespace std;

int main( ) {
    cout << "C++ Tutorial";
    cout << " Java language"<<endl;
    cout << "End of line"<<endl;
}
```



Introduction

Variable

- A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.

syntax to declare a variable:

type **variable_list**;

Example:

int **x**;

float **y**;

char **z**;

Here, x, y, z are variables and int, float, char are data types.



Introduction

Variable cont

- We can also provide values while declaring the variables as given below:

```
int x=5,b=10; //declaring 2 variable of integer type
```

```
float f=30.8;
```

```
char c='A';
```




Introduction

Rules for defining variables

1. A variable can have alphabets, digits and underscore.
2. A variable name can start with alphabet and underscore only. It can't start with digit.
3. No white space is allowed within variable name.
4. A variable name must not be any reserved word or keyword e.g. char, float etc.



Introduction

Rules for defining variables cont

Valid variable names:

```
int a;  
int _ab;  
int a30;
```

InValid variable names:

```
int 4;  
int x y;  
int double;
```



Introduction

Data Types

There are 4 types of data types

Types	Data Types
Basic Data Type	int, char, float, double, etc
Derived Data Type	array, pointer, etc
Enumeration Data Type	enum
User Defined Data Type	structure



Introduction

Data Types

Basic Data Types

- The basic data types are integer-based and floating-point based.
- C++ language supports both signed and unsigned literals.
- The memory size of basic data types may change according to 32 or 64 bit operating system.
- Let's see the basic data types. It size is given according to 32 bit OS.



Introduction

Data Types

Data Types	Memory Size	Range
char	1 byte	-128 to 127
signed char	1 byte	-128 to 127
unsigned char	1 byte	0 to 127
short	2 byte	-32,768 to 32,767
signed short	2 byte	-32,768 to 32,767
unsigned short	2 byte	0 to 32,767
int	2 byte	-32,768 to 32,767
signed int	2 byte	-32,768 to 32,767
unsigned int	2 byte	0 to 32,767
short int	2 byte	-32,768 to 32,767
signed short int	2 byte	-32,768 to 32,767
unsigned short int	2 byte	0 to 32,767



Introduction

Operators

An operator is simply a symbol that is used to perform operations

The following are types of operators to perform different types of operations.

- ❖ Arithmetic Operators
- ❖ Relational Operators
- ❖ Logical Operators
- ❖ Bitwise Operators
- ❖ Assignment Operator
- ❖ Unary operator
- ❖ Ternary or Conditional Operator
- ❖ Misc Operator



Introduction

Identifiers

- Identifiers in a program are used to refer to the name of the variables, functions, arrays, or other user-defined data types created by the programmer.
- Every language has its own rules for naming the identifiers.



Introduction

Rules that are common in both C and C++

1. Only alphabetic characters, digits, and underscores are allowed.
2. The identifier name cannot start with a digit, i.e., the first letter should be alphabetical. After the first letter, we can use letters, digits, or underscores.
3. In C++, uppercase and lowercase letters are distinct. Therefore, we can say that C++ identifiers are case-sensitive.
4. A declared keyword cannot be used as a variable name.



Introduction

Rules that are common in both C and C++

The following are the examples of valid identifiers are:

1. Result
2. Test2
3. _sum
4. power



Introduction

Rules that are common in both C and C++

The following are the examples of invalid identifiers:

Sum-1 // containing special character '-'.

2data // the first letter is a digit.

break // use of a keyword.



Introduction

simple example to understand the concept of identifiers.

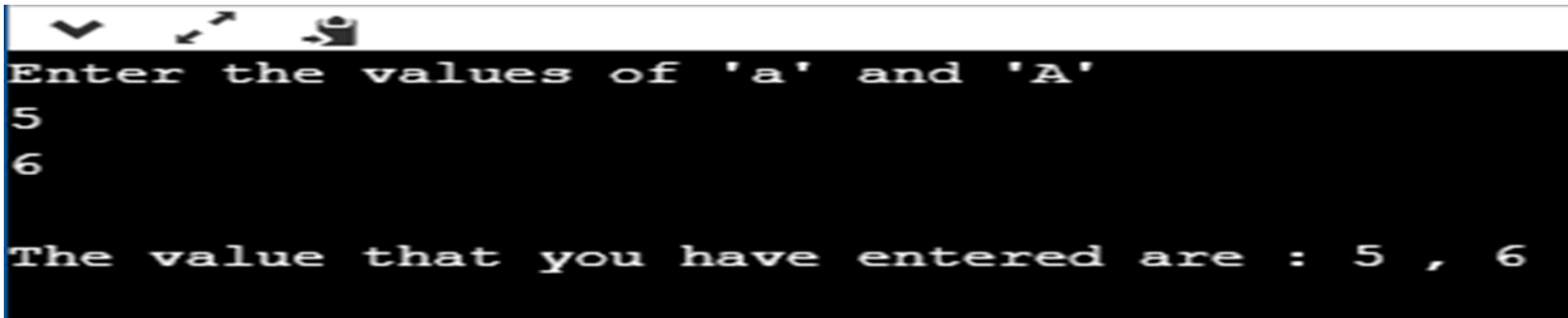
```
#include <iostream>
using namespace std;
int main()
{
    int a;
    int A;
    cout<<"Enter the values of 'a' and 'A'";
    cin>>a;
    cin>>A;
    cout<<"\nThe values that you have entered are : "<<a<<" , "<<A;
    return 0;
}
```



Introduction

- In the above code, we declare two variables 'a' and 'A'. Both the letters are same but they will behave as different identifiers.
- As we know that the identifiers are the case-sensitive so both the identifiers will have different memory locations.

Output

A screenshot of a terminal window with a black background and white text. At the top, there is a prompt "Enter the values of 'a' and 'A'". Below this, the user has entered "5" on the first line and "6" on the second line. At the bottom, the program has printed the output "The value that you have entered are : 5 , 6".

```
Enter the values of 'a' and 'A'
5
6
The value that you have entered are : 5 , 6
```



Introduction

Expression

- Expression consists of operators, constants, and variables which are arranged according to the rules of the language.
- It can also contain function calls which return values. An expression can consist of one or more operands, zero or more operators to compute a value.

Examples of C++ expression:

1. $(a+b) - c$
2. $(x/y) - z$
3. $4a^2 - 5b + c$
4. $(a+b) * (x+y)$



Introduction

Expression

An expression can be of following types:

1. Constant expressions
2. Integral expressions
3. Float expressions
4. Pointer expressions
5. Relational expressions
6. Logical expressions
7. Bitwise expressions
8. Special assignment expressions



Introduction

Simple program containing constant expression:

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int x;    // variable declaration.
```

```
    x=(3/2) + 2; // constant expression
```

```
    cout<<"Value of x is : "<<x; // displaying the value of x.
```

```
    return 0;
```

```
}
```

- In the above code, we have first declared the 'x' variable of integer type. After declaration, we assign the simple constant expression to the 'x' variable.



Introduction

Simple example of integral expression:

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int x; // variable declaration.
```

```
    int y; // variable declaration
```

```
    int z; // variable declaration
```

```
    cout<<"Enter the values of x and y";
```

```
    cin>>x>>y;
```

```
    z=x+y;
```

```
    cout<<"\n"<<"Value of z is :"<<z; // displaying the value of z.
```

```
    return 0;
```

```
}
```




ASSIGNMENT

Write the program for the following expression

1. Float expression
2. Pointer expression

Thank you for Listening! Questions

